

AMOEBA: Adaptive Multi-Task Execution on Hierarchical Spatial Dataflow Architectures

HPCA 2027 Submission #NaN

Confidential Draft

Do NOT Distribute!!

Abstract—Coarse-Grained Reconfigurable Architectures (CGRAs) are a promising and versatile accelerator platform, offering a balance between the performance and efficiency of specialized accelerators and software programmability. However, their full potential is severely hindered by control flow in accelerated kernels, as control flow (e.g., loops, branches) is fundamentally incompatible with the parallel, data-driven CGRA fabric. Prior strategies to resolve this mismatch in CGRA kernel acceleration are either inefficient, sacrificing performance for generality, or lack generality due to the difficulty of adapting them across different execution models. Thus, a general and unified solution for efficient CGRA kernel acceleration remains elusive. This paper introduces NEURA, a unified and retargetable compilation framework that systematically resolves the control-dataflow mismatch in CGRAs. NEURA’s core innovation is a novel, pure dataflow intermediate representation (IR) built on a predicated type system. In this IR, control contexts are embedded as a predicate within each data, making control an intrinsic property of data. This mechanism enables NEURA to systematically flatten complex control flow into a single unified dataflow graph. This unified representation decouples kernel representation from hardware, empowering NEURA to retarget diverse CGRAs with different execution models and microarchitectural features.

I. INTRODUCTION

Spatial Dataflow Architectures (SDAs) [9], [10], [12], [14], [19], [26], [40], [49], [53], such as Coarse-Grained Reconfigurable Arrays (CGRAs), offer a compelling balance between the efficiency of fixed-function accelerators and the flexibility of general-purpose processors (e.g., CPUs and GPUs) [28], [32], driving their adoption across domains like machine learning (ML) [21], [22], [36] and high-performance computing (HPC) [17], [54]. As workloads in these domains increasingly consist of multiple interacting tasks, scaling a monolithic SDA into a larger flat array leads to prohibitive mapping complexity and resource fragmentation [20]. Recent SDAs therefore adopt a hierarchical organization, grouping compute tiles into modular cores and connecting multiple cores through an inter-core fabric [5], [8], [31], [34], [35], [45]. We refer to this class of architectures as hierarchical SDAs.

As shown in Fig. 1, a hierarchical SDA typically consists of multiple spatially distributed cores interconnected through an inter-core Network-on-Chip (NoC) [35], [39], [45]. Within each core, a mesh of compute tiles is interconnected by flexible routing resources, such as crossbars, to form a highly configurable tile array [19]. To execute an application on such a fabric, the compiler represents the workload using abstractions such as Control-Data Flow Graphs (CDFGs) [31],

[57] and task graphs [5], and orchestrates computations across both inter-core resources and intra-core tile arrays.

We observe that multi-task applications impose heterogeneous demands on hierarchical SDA resources. First, different tasks demand different numbers of tiles. Within each task, a Dataflow Graph (DFG) represents operations as nodes and data dependences as edges. Mapping a small DFG onto a large tile array leaves many tiles underutilized, while mapping a large DFG onto a small tile array makes execution resource-bound and prevents the task from fully exploiting its instruction-level parallelism (ILP). Second, different tasks demand different numbers of cores. Some tasks can replicate their configurations across multiple cores and process partitioned data in parallel for data-level parallelism (DLP), but assigning too few cores to such tasks can create task-pipeline bottlenecks, while assigning too many cores wastes resources that could otherwise serve other tasks. Third, the task graph of an application demands task-level spatio-temporal scheduling over the core array. Independent tasks and pipeline stages require cores concurrently to exploit task-level parallelism, but when their aggregate resource demand exceeds the physical fabric, cores must be reused over time. Failing to adapt to these heterogeneous demands of multi-task applications leads to resource underutilization and limits the parallelism that hierarchical SDAs can exploit.

However, existing hierarchical SDA frameworks are not designed around adapting resources to these heterogeneous demands. They primarily use task graphs to expose task-level parallelism across distributed cores [5], [8], [31], [34], [57]. Yet these frameworks typically schedule tasks at a fixed granularity, often assuming one core per task, while overlooking each task’s demand for the number of tiles and the number of cores. As a result, a task may be constrained by the tile array within one core, or may miss opportunities to use additional cores for configuration replication and higher DLP. Moreover, most hierarchical SDA frameworks mainly emphasize spatial placement of tasks across distributed cores, leaving efficient temporal resource reuse under limited physical resources insufficiently supported. Some works consider temporal reuse by partitioning task graphs into temporally multiplexed blocks [7] or by dynamic scheduling irregular tasks over time [5], [31], but they still do not adapt the resources assigned to each task according to its tile and core demands. Consequently, existing hierarchical SDA frameworks struggle to satisfy the heterogeneous tile, core, and task-level spatio-temporal demands of

multi-task applications.

To address these limitations, we propose **AMOEBa** — a novel framework supporting adaptive multi-task execution on hierarchical spatial dataflow architectures. AMOEBa adapts hierarchical SDA resources to the heterogeneous tile, core, and task-level spatio-temporal demands of multi-task applications through a coordinated representation, architecture, runtime, and compiler stack. Instead of scheduling tasks over fixed physical granularities, AMOEBa makes application resource demands explicit and adapts the underlying resources accordingly. The specific contributions are as follows:

- We introduce a resource-aware hierarchical data-driven representation for multi-task applications (Sec. III). It captures both inter- and intra-task execution while preserving structured information for compiler analysis of hierarchical SDA resource demands;
- We design architecture and runtime support for adaptive multi-task execution on hierarchical SDAs (Sec. IV). AMOEBa can compose neighboring tile arrays for heterogeneous tile demands, and supports static and dynamic core allocation for data-partitioned execution to satisfy heterogeneous core demands;
- We develop compiler support for resource-aware task scheduling on hierarchical SDAs (Sec. V). The compiler analyzes tile usage, core allocation, and task-level spatio-temporal scheduling, and generates partition templates for runtime task replication and pipeline rebalancing;
- We conduct comprehensive evaluations of AMOEBa

xxxxx

II. BACKGROUND AND MOTIVATION

This section begins by using a simplified attention layer as an example to illustrate how a multi-task application executes on a hierarchical SDA in Sec. II-A. We then use profiling results and scheduling examples in Sec. II-B to show that multi-task applications impose heterogeneous demands on hierarchical SDA resources. Finally, Sec. II-C summarizes the key challenges toward adaptive multi-task execution on hierarchical SDAs that motivate the design of AMOEBa.

A. Multi-Task Execution on Hierarchal SDAs

Fig. 1 illustrates how a simplified attention layer [52] executes on a hierarchical SDA. The attention layer is represented as a multi-task application consisting of three projection tasks for Q , K , and V , followed by attention score computation, softmax, and value aggregation. These tasks form a task graph that captures inter-task dependences, while each task internally contains a DFG that captures its computation.

Task-Graph Execution across Distributed Cores. As shown in Fig. 1(a), hierarchical SDAs exploit task-level parallelism by scheduling tasks in the task graph across distributed cores [5], [7], [8], [57]. Tasks without data dependences can execute concurrently (e.g., Tasks 0, 1, and 2), while dependent tasks can form a task-level pipeline as their inputs become available (e.g., Task 4 can start once the outputs of Task 3 are ready). When the number of tasks exceeds the available cores,

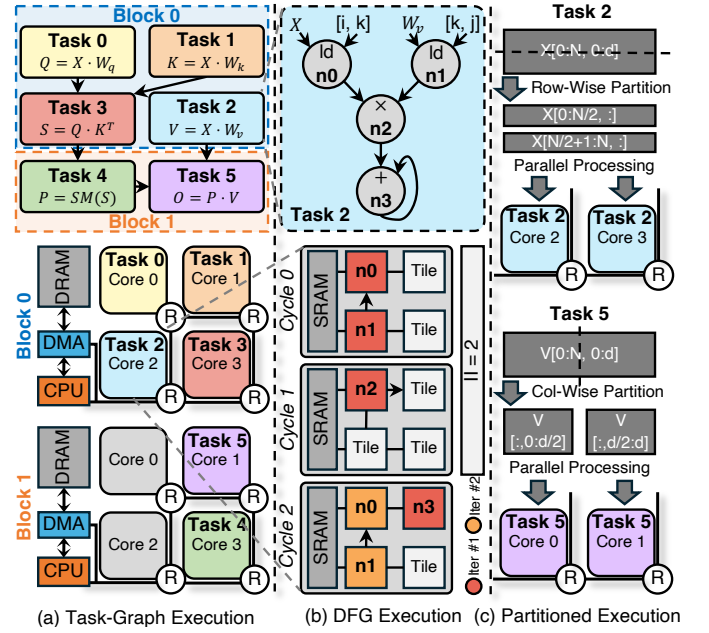


Fig. 1. Multi-Task Execution on Hierarchical SDAs — (a) The attention task graph is mapped onto distributed cores and split into temporal task blocks when the number of tasks exceeds the available cores. (b) The DFG of a projection task is executed on a core's tile array using modulo scheduling. (c) DLP-capable tasks replicate the same configuration across cores and process row-wise or column-wise data partitions in parallel.

existing works [7], [45] often group tasks into temporally multiplexed blocks, so that different groups of tasks reuse the core array over time. This coarse-grained multiplexing enables task-graph execution under limited resources.

DFG Execution on a Tile Array. Within each task, computation is represented as a DFG and mapped onto the tile array of a core. Fig. 1(b) shows Task 2 as an example. The compiler maps the DFG onto the tile array using modulo scheduling [29], [37], which pipelines loop iterations by initiating a new iteration every initiation interval (II) cycles. The resulting schedule overlaps multiple loop iterations on the tile array, allowing each task to exploit ILP within a core.

Data-Partitioned Execution across Cores. Fig. 1(c) shows that some tasks expose DLP when their input or output data can be partitioned across cores, enabling multiple task replicas to execute in parallel. For Task 2, one possible partitioning splits the input matrix along the sequence dimension (i.e., row-wise partitioning), so each core computes a separate row range of the value projection. For Task 5, one possible partitioning splits the value matrix along the feature dimension (i.e., column-wise partitioning), so each core computes a separate output-column range during value aggregation. In both examples, the same task configuration is replicated across cores, while each replica processes a different data partition. This allows hierarchical SDAs to use additional cores to improve the throughput of DLP-capable tasks.

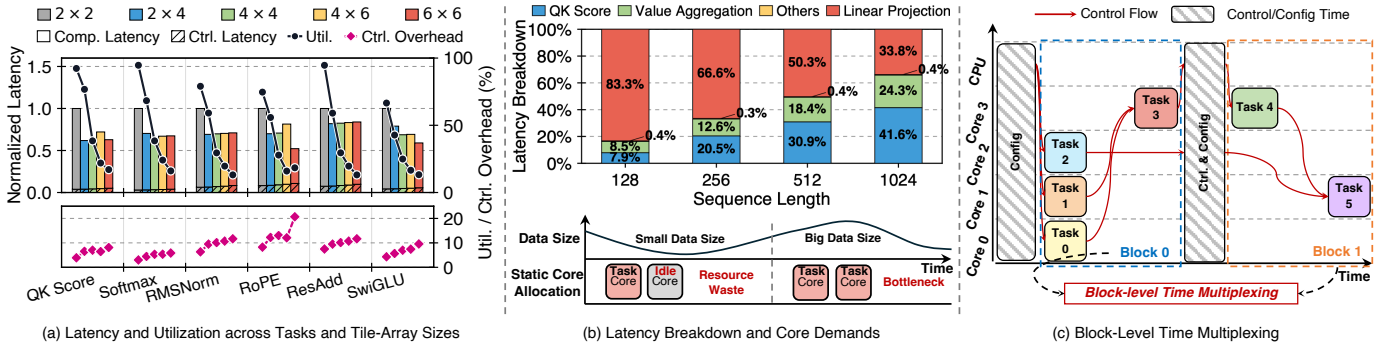


Fig. 2. Heterogeneous Resource Demands on Hierarchical SDAs — (a) Representative LLaMA2-7B tasks are profiled on a conventional SDA with CPU-managed loop control across tile-array sizes. Latencies are normalized to each task’s latency on a 2×2 tile array, and control overhead is defined as control latency/total latency. (b) LLaMA2-7B latency breakdown profiled under one-core-per-task execution across different sequence lengths, with a static core allocation example illustrating resource waste and bottlenecks under different data sizes. (c) Task-graph execution with block-level time multiplexing.

B. Heterogeneous Resource Demands

Although hierarchical SDAs provide multiple levels of resources, multi-task applications impose heterogeneous demands on these resources. As shown in Fig. 2, different tasks require different tile-array sizes and different numbers of cores, while the task graph further demands task-level spatio-temporal scheduling under limited core resources.

Different Tile Demands across Tasks. Fig. 2(a) profiles representative tasks from LLaMA2-7B [50] on a conventional single-core SDA [47] and shows that different tasks exhibit different latency and utilization across tile-array sizes. For example, QK Score and Softmax achieve the lowest latency on a 4×4 tile array, whereas RoPE and SwiGLU perform best on a 6×6 tile array. Larger arrays provide more spatial mapping opportunities when a DFG has sufficient parallelism. However, executing a DFG on an overly large array can cause low utilization and performance degradation due to the increased mapping complexity from a larger exploration space (e.g., RMSNorm). Therefore, a fixed tile-array size either constrains tasks that need more tiles or wastes resources on tasks that need fewer tiles.

Different Core Demands across Tasks. Fig. 2(b) profiles the latency breakdown of LLaMA2-7B under different sequence lengths, with each task using one core. The profiling shows that DLP-capable tasks contribute unevenly to latency under the same one-core-per-task allocation. For example, QK Score, Value Aggregation, and Linear Projection have different latency contributions. Since these tasks can reduce latency by using additional cores through data-partitioned execution, assigning more cores to bottleneck tasks can better balance the task pipeline.

Task-Level Spatio-Temporal Scheduling Demands. When the task graph contains more tasks than the available cores, tasks must be placed across cores and reused over time. This temporal reuse becomes difficult when tasks have different tile and core demands, because the scheduler must decide when a task reuses resources while considering its resource demands. Some works group tasks into temporally multiplexed

blocks [7], [45]. As shown in Fig. 2(c), tasks are spatially scheduled within each task block, and resources are reused through block switching. However, such block-level reuse introduces control and configuration intervals that interrupt task-level pipelining across blocks. Other works dynamically schedule irregular tasks over time [5], [31], but their scheduling still operates over fixed resource granularity. Therefore, adaptive multi-task execution on hierarchical SDAs requires task-level spatio-temporal scheduling coordinated with each task’s resource demands.

C. Challenges Towards Adaptive Multi-Task Execution

As introduced in Sec. II-B, multi-task applications impose heterogeneous resource demands when executed on hierarchical SDAs. Satisfying these demands requires hierarchical SDA frameworks to adjust resources according to each task’s needs, enabling adaptive multi-task execution. We identify three key challenges in supporting adaptive execution from the representation, architecture, and compiler perspectives.

Challenge 1: Exposing Resource Demands in Multi-Task Representation. Adaptive multi-task execution first requires a representation that exposes resource-relevant structure from multi-task applications. Some works introduce task graphs to capture inter-task dependences [5], [7], [57], but they do not expose the intra-task computation needed to infer tile and core demands. In addition, representations that encode task dependences through specialized communication primitives [41], [57] make it difficult to support task-level spatio-temporal scheduling. On the other hand, DFGs [14], [29], [47] expose intra-task computation and are suitable for analyzing tile demand, but they do not naturally expose core demand for data-partitioned execution across cores.

Challenge 2: Supporting Heterogeneous Resource Demands in Architecture. Even with a resource-aware representation, hierarchical SDAs need architectural support to satisfy heterogeneous resource demands. For tile demands, existing frameworks often assume fixed tile arrays [5], [8], [35], [39], [57], which cannot adapt the tile-array size to each task. Although Neura [45] supports composing neighboring cores into

TABLE I

COMPARISON OF AMOEBA WITH EXISTING HIERARCHICAL SDA FRAMEWORKS. ●— FULLY SUPPORTED; ◐— PARTIALLY SUPPORTED; ○— NOT SUPPORTED. PARTIAL SUPPORT INDICATES A FEATURE IS NOT FULLY SUPPORTED OR THE EXISTING DESIGN CAN BE EASILY EXTENDED TO SUPPORT IT.

Framework	Resource-Aware Multi-Task Representation	Architecture / Runtime Support				Compiler Support	
		Adaptive Tile-Array Size	Distributed & Hierarchical Loop Control	Dynamic Core Allocation	Task-Level Spatio-Temporal Exec.	Automatic Resource Analysis	Joint Tile/Core/Time Scheduling
SARA [57]	◐	○	◐	○	○	◐	○
TaskStream [5]	◐	○	○	◐	◐	○	◐
Neura [45]	○	●	○	○	◐	◐	○
Octopus [8]	◐	○	○	◐	◐	◐	◐
Fifer [31]	○	○	○	○	◐	◐	◐
SN40L [34]	◐	○	◐	○	○	◐	○
STeP [41]	◐	○	◐	◐	◐	○	◐
TRIPS [39]	○	◐	○	○	○	○	○
AMOEBA (Ours)	●	●	●	●	●	●	●

a larger tile array, it still uses the conventional CPU-managed approach for control flow (e.g., nested loops). As shown at the bottom of Fig. 2(a), this centralized control introduces non-negligible control overhead, especially on larger tile arrays. For core demands, the effective number of replicas for a DLP-capable task may change with runtime input sizes. Fig. 2(b) shows that DLP-capable tasks contribute differently to latency across sequence lengths. A static core allocation can waste cores at small data sizes or create pipeline bottlenecks at large data sizes. Hierarchical SDAs need runtime support to adjust the number of cores assigned to DLP-capable tasks. For task-level spatio-temporal demands, architecture support must go beyond block-level time multiplexing to avoid interrupting the task-level pipeline and enable task-level spatio-temporal execution while accounting for different task demands.

Challenge 3: Scheduling Tasks across Tiles, Cores, and Time. With resource-aware representation and architectural support, the compiler must translate heterogeneous resource demands into concrete schedules. Prior compilers typically optimize one level, such as mapping a DFG onto a tile array [13], [14], [19], [24] or scheduling tasks across cores at a fixed granularity [5], [7], [8], [57]. This is insufficient for adaptive multi-task execution, where the compiler must jointly decide how many tiles each task uses, how many cores are assigned to each task, and which tasks reuse resources over time. These decisions are tightly coupled. Using more tiles for one task can reduce resources for other tasks, assigning more cores to a bottleneck task can change the task-level pipeline balance, and temporal reuse must account for the different resource demands of tasks sharing the same cores over time.

A comparison between AMOEBA and existing hierarchical SDA frameworks is summarized in Table I. AMOEBA first introduces a resource-aware multi-task representation that exposes inter-task dependences, per-task DFG structure, and data-partitioning opportunities, addressing **Challenge 1** by making heterogeneous resource demands analyzable (see Sec. III). AMOEBA then provides architecture and runtime support for composing tile arrays with distributed and hierarchical loop control, supporting static and dynamic core allocation for data-partitioned execution, and enabling task-level spatio-temporal execution, addressing **Challenge 2** (see Sec. IV). Finally, AMOEBA introduces a compiler that automatically constructs the resource-aware representation and

schedules tasks across tiles, cores, and time, addressing **Challenge 3** (see Sec. V).

III. RESOURCE-AWARE MULTI-TASK REPRESENTATION

AMOEBA introduces a resource-aware multi-task representation to expose the structures needed for adaptive multi-task execution. As shown in Fig. 3, the representation is hierarchical, combining an inter-task data-driven representation with an intra-task data-driven representation. Sec. III-A first describes how AMOEBA uses these two levels of representation to model multi-task applications. Sec. III-B then explains how this representation exposes heterogeneous resource demands for adaptive multi-task execution.

A. Hierarchical Data-Driven Representation

AMOEBA represents multi-task applications using a hierarchical data-driven representation, which consists of an inter-task representation for task-level execution and an intra-task representation for computation within each task.

Inter-Task Data-Driven Representation. At the inter-task level, AMOEBA represents an application as a task graph. Each node is a task, and each edge represents a dependence between two tasks. AMOEBA distinguishes two types of edges: (1) **Value Dependence Edges** carry values from a producer or input source to a consumer task, such as the sequence length L in Fig. 3; and (2) **Memory Dependence Edges** capture inter-task dependences through memory objects, such as the dependence through $Q[L, D]$ in Fig. 3. A task becomes ready when its required input dependences are satisfied. After execution, it emits output values and memory-dependence notifications along its outgoing edges, allowing successor tasks to fire through peer-to-peer data-driven signaling. This representation allows independent tasks to fire concurrently, while dependent tasks fire as their inputs become available, forming task-level pipelines. By representing task dependences explicitly rather than encoding them through specialized communication primitives, AMOEBA naturally supports task-level spatio-temporal scheduling.

Intra-Task Data-Driven Representation. Within each task, AMOEBA represents computation as a DFG, where nodes are operations and edges represent data dependences between operations. Since a task often executes the same DFG over a regular iteration domain, AMOEBA makes such repeated execution explicit using counter-driven execution. AMOEBA

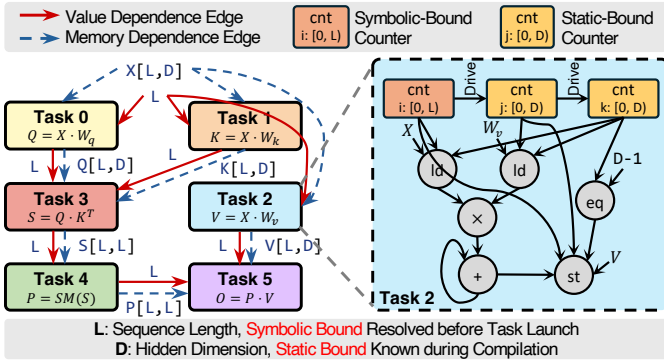


Fig. 3. Resource-Aware Multi-Task Representation — A simplified attention layer is represented as an inter-task data-driven representation with value and memory dependence edges. Task 2 is expanded into an intra-task DFG driven by static-bound and symbolic-bound counters.

distinguishes two types of counters: (1) **Static-Bound Counters** have compile-time constant bounds, such as the hidden dimension D ; and (2) **Symbolic-Bound Counters** use symbolic bounds that are resolved before task launch, such as the sequence length L . These counters can form a parent-child chain to drive nested iterations, as shown by the counters of Task 2 in Fig. 3. For dynamic data-driven execution that does not fit the counter-driven form, where the iteration bound or termination condition is determined during task execution, AMOEBA flattens the control flow into general dataflow through predication [15], [24].

B. Exposing Heterogeneous Resource Demands

The intra-task representation exposes DFG structure and partitionable counter domains for tile/core-demand analysis, while the inter-task representation keeps tasks and readiness dependences explicit for task-level spatio-temporal scheduling.

Tile Demand is exposed by the intra-task DFG, which captures each task's operations and data dependences. For example, by profiling the mapping results of a DFG under different tile-array sizes, the compiler can reason about how many tiles the task can effectively use.

Core Demand is exposed by combining counter domains with the DFG's data dependences and memory accesses. A counter domain provides a candidate iteration domain for partitioning, while data dependences and memory accesses determine whether the partitioned ranges can be safely executed across cores. If a counter domain can be split into disjoint ranges without violating data dependences or causing memory access conflicts, multiple replicas of the same task can process different ranges on different cores. For example, counter i of Task 2 in Fig. 3 exposes a candidate partitioning dimension, and the task's data dependences and memory accesses indicate that Task 2 can be partitioned along this iteration domain. By marking counters as either static-bound or symbolic-bound, AMOEBA further exposes whether the partition range is known at compile time or resolved before task launch, enabling runtime core allocation (see Sec. IV-C).

Task-Level Spatio-Temporal Scheduling Opportunities are exposed by keeping each task as an independent scheduling unit and representing inter-task dependences explicitly. These dependences specify when a task can fire, but do not prescribe where the task must execute or which communication primitive must be used between tasks. Therefore, the scheduler can decide which ready tasks execute concurrently and which tasks reuse cores over time while preserving inter-task dependences.

IV. AMOEBA ARCHITECTURE AND RUNTIME

AMOEBA provides architecture and runtime support to satisfy the heterogeneous resource demands exposed by its representation. Sec. IV-A first introduces the hierarchical organization of AMOEBA architecture. Sec. IV-B and Sec. IV-C then present elastic tile-array composition for tile demand and adaptive core allocation for core demand, respectively. Finally, Sec. IV-D explains how context-based task execution enables task-level spatio-temporal execution.

A. Hierarchical Architecture Organization

Fig. 4 gives an overview of the AMOEBA architecture. AMOEBA is loosely coupled with a central processing unit (CPU) and is invoked through the accelerator command interface. A direct memory access unit (DMA Unit) loads data into scratchpad memories (SPMs) and configurations into context memories of each core, respectively. The runtime module monitors resource status and sends commands for dynamic task replication and core allocation. The core array contains multiple cores connected by an inter-core mesh NoC for data, dependence notifications, and control signals. Each core integrates a tile array, SPM, loop controller, and core controller. The SPMs across cores form a global address space, allowing a core to access data from other cores through the inter-core NoC. Tiles inside a core are connected by crossbars and communicate with controllers through a ring NoC. Following conventional CGRA execution [25], [47], a task DFG is mapped onto a tile array in a modulo-scheduled manner [29]. Each tile contains functional units, register files, configuration memory, a routing crossbar, and a distributed counter unit (DCU). The DCUs work with the loop controller to drive counter-driven DFG execution.

B. Elastic Tile-Array Composition for Tile Demand

Different tasks require different tile demands, yet simply composing physical tile arrays into a larger tile array can introduce non-negligible loop-control overhead, as shown in Fig. 2(a). AMOEBA satisfy different tile demands by supporting both single-array and composed-array execution while using hierarchical and distributed loop control to amortize the control overhead on larger tile arrays.

Tile-Array Composition. AMOEBA enables tile-array composition by connecting the boundary tiles of each core with the tile array of neighboring cores through crossbars, as shown in Fig. 4(b). These cross-core tile connection allow values to be routed across the core boundary, so tiles from adjacent tile-arrays can jointly execute one task DFG. As shown

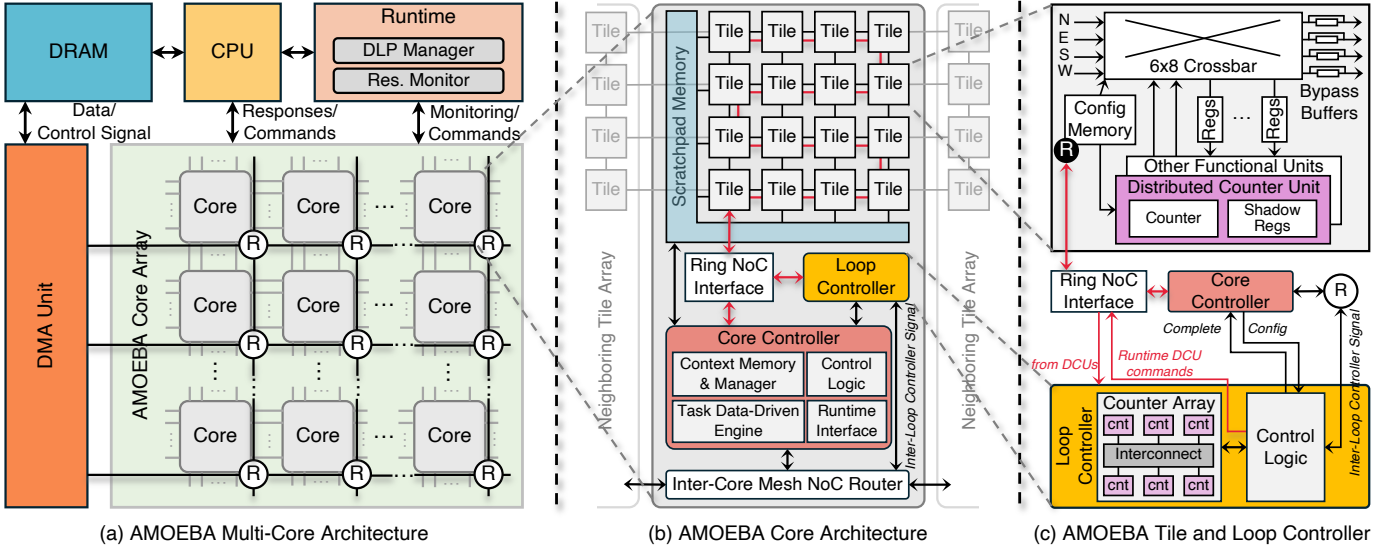


Fig. 4. AMOEBA Architecture Overview — (a) AMOEBA multi-core array is loosely coupled with a CPU, DRAM, DMA unit, and runtime module, and its cores are connected through an inter-core mesh NoC. (b) Each core integrates a tile array, scratchpad memory, loop controller, core controller, context memory, task data-driven engine, and runtime interface. (c) Each tile contains a distributed counter unit (DCU) for inner-loop counters, while the loop controller maintains outer-loop counters and coordinates composed-array execution through inter-loop-controller signals.

in Fig. 5(a), a task can be mapped either to a single 4×4 tile array or to a composed 8×4 tile array formed by neighboring cores. For composed-array execution, the compiler treats the participating tiles as one mapping target, allowing a task DFG with higher tile demands to be mapped across multiple cores.

Hierarchical and Distributed Loop Control. AMOEBA provides hardware support for the static-bound and symbolic-bound counter semantics introduced in Sec. III-A. As shown in Fig. 4(c), each tile integrates a distributed counter unit (DCU) that executes the innermost loop counter and drives DFG execution. The DCU also stores current outer-loop indices in shadow registers, allowing DFG operations to directly use these indices without repeatedly querying the loop controller. Since the innermost loop counter updates most frequently, executing it in each DCU avoids interacting with the loop controller or CPU for every innermost-counter update. The loop controller maintains outer-loop counters through a configurable counter array. When a DCU completes the current innermost-loop range, it sends a complete signal to the loop controller. The loop controller then advances the outer-loop counters, determines whether the innermost loop should be reset, and updates DCU shadow registers with the new outer-loop indices. For composed-array execution, loop controllers of neighboring cores coordinate outer-loop updates through inter-loop-controller signals and logically form one loop controller for the composed tile array. This peer-to-peer coordination avoids centralized CPU-managed loop control and amortizes control overhead on larger composed tile arrays.

C. Adaptive Core Allocation for Core Demand

DLP-capable tasks expose different core demands when executed in a data-partitioned manner across cores. AMOEBA

supports both static and dynamic core allocation by executing each DLP-capable task as one or more task replicas, where the replica count is determined by the compiler for static cases and adjusted by the runtime for dynamic cases.

Data-Partitioned Execution Support. AMOEBA implements data-partitioned execution by replicating a DLP-capable task’s configuration across cores and assigning each task replica a disjoint partition. For each replica count, the compiler generates a partition configuration that specifies the counter bounds and memory regions for each task replica. Therefore, all replicas execute the same DFG while processing disjoint portions of data. For static core demand, the compiler determines the replica count and the corresponding partition configuration. For dynamic core demand, the runtime adjusts the replica count and applies the corresponding partition configuration. The task is considered complete only after all replicas of the current execution finish, preserving the original inter-task dependence semantics.

Throughput-Guided Task Pipeline Rebalancing. AMOEBA records the latency of every task over a profiling time window and performs task pipeline rebalancing at window boundaries. As shown in Fig. 5(b), the runtime identifies the bottleneck task as the task with the largest latency on the pipeline path. Replica-count adjustment is triggered only when the bottleneck task is a compiler-identified DLP-capable task. To decide where to store the new replica configuration, AMOEBA evaluates each core with an unused context-memory entry and estimates the path output interval after assigning the new replica to that core. If the best core reduces the output interval, AMOEBA increases the bottleneck task’s replica count by one and applies the compiler-generated partition configuration for the new replica

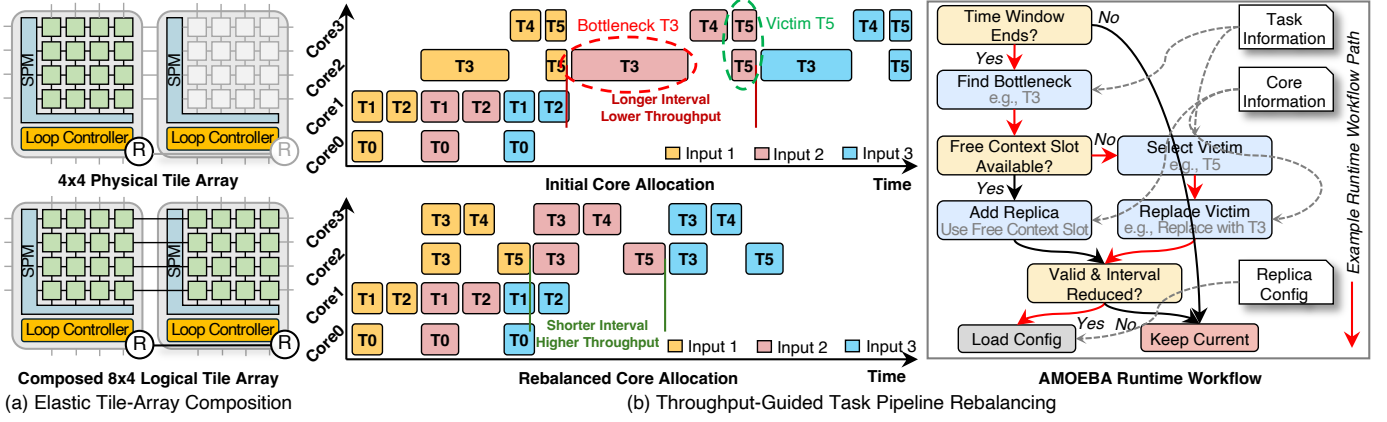


Fig. 5. Elastic Tile-Array Composition and Runtime Pipeline Rebalancing — (a) Neighboring tile arrays can execute independently as 4×4 arrays or be composed into a larger 8×4 logical tile array. (b) The runtime rebalances task replicas according to windowed latency profiling. It evaluates available cores and victim replicas using the output interval, then loads the updated task configurations into the corresponding context memory.

count. If no available core is beneficial, the runtime identifies the runtime-managed DLP-capable task with the smallest latency and more than one replica on the same pipeline path as the victim task. If no such victim task exists, AMOEBA keeps the current replica allocation. Otherwise, for each replica of the victim task, AMOEBA estimates the output interval after replacing that replica’s context-memory entry with the new bottleneck-task replica configuration. AMOEBA then applies the best update only if it reduces the path output interval.

Interaction with Task Pipeline. AMOEBA does not stop the task pipeline during rebalancing. Pipeline rebalancing is performed by the runtime, which updates the replica allocation and prepares the adjusted task configurations using the compiler-generated partition configurations. Configuration loading occurs only when the target context-memory entry is idle, so it may delay the next launch of the affected task or dependent task executions, but does not interrupt tasks that are already running on the same core or other cores. The loaded configuration takes effect only when the corresponding task replica is launched in its next execution.

D. Context-Based Task-Level Spatio-Temporal Execution

AMEOBA supports task-level spatio-temporal execution through context-based execution, where task configurations are stored in each core’s context memory and activated according to data-driven readiness.

Context-Based Task Configuration Management. As shown in Fig. 4(b), each core contains a context memory, where each context-memory entry stores one context corresponding to the compiler-generated configuration of one task or one task replica. A context includes the tile-array configuration and loop-controller configuration needed to execute the task on that core. In our implementation, each core provides three context-memory entries, so at most three task or replica configurations can be stored in one core at the same time. For replicated tasks, the runtime records which cores store the

configuration of their replicas and updates these records when replica counts change.

Data-Driven Spatio-Temporal Context Activation. The task data-driven engine tracks dependence readiness and activates a context when its input dependences are satisfied and its core is available. When a task execution completes, it sends readiness notifications to the cores that store its consumer task configurations through the inter-core NoC. AMOEBA uses credit-based flow control [5], [6] for these dependence packets to handle producer-consumer rate mismatches in the task pipeline. Upon receiving the notifications, the corresponding task data-driven engines update the readiness status of the target contexts. Contexts on different cores can be activated independently and execute concurrently, enabling spatial task execution. Contexts sharing the same core are activated over time according to a compiler-generated order that follows pipeline dependences. For example, T1 and T2 in Fig. 5 share one core. AMOEBA activates T1 before T2 to avoid deadlock during temporal reuse. For tasks using a composed tile array, the engines activate the context only after dependences are satisfied and synchronization signals from neighboring participating cores are received.

V. AMOEBA COMPILATION FRAMEWORK

Fig. 6 gives an overview of the AMOEBA compilation framework. AMOEBA takes programs written in PyTorch [2] or C/C++ and lowers them into structured MLIR through Torch-MLIR [48] and Polygeist [30]. The compiler then constructs a resource-aware multi-task representation, classifies and profiles each task, and orchestrates task execution on the AMOEBA architecture. Sec. V-A explains how AMOEBA extracts tasks, DFGs, and inter-task dependences from the lowered MLIR [23]. Sec. V-B describes how the compiler identifies DLP-capable tasks and profiles tile-array candidates. Sec. V-C presents throughput-guided task orchestration. This stage generates configurations for execution.

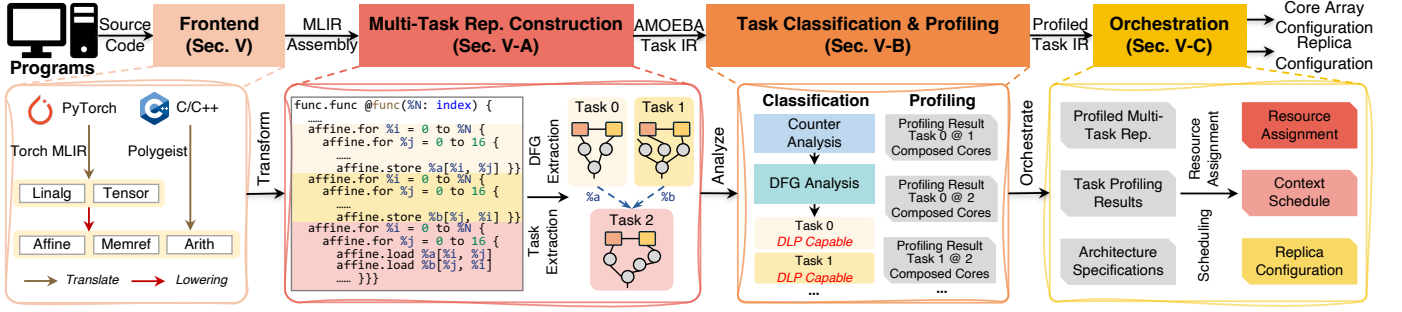


Fig. 6. AMOEBA Compilation Framework — AMOEBA lowers input programs into structured MLIR, constructs and profiles a resource-aware multi-task representation, and performs throughput-guided orchestration to generate resource assignments, context schedules, and runtime replica configurations.

A. Resource-Aware Multi-Task Representation Construction

AMOEBA first converts the lowered MLIR program into AMOEBA task intermediate representation (IR), a resource-aware multi-task representation implemented in MLIR. In the current implementation, each top-level affine loop nest is extracted as one task, which matches the coarse-grained pipeline stages in our evaluated workloads. This representation can also accept task boundaries provided by other frontends or programmer annotations. When forming each task, AMOEBA records the memory objects read and written by the loop nest, together with scalar values that are defined outside the loop body. These task inputs and outputs are used to preserve read-after-write (RAW), write-after-write (WAW), and write-after-read (WAR) dependences across tasks. Within each task, AMOEBA recognizes perfectly nested affine loops and converts their loop structure into an explicit counter hierarchy, preserving loop bounds, steps, and nesting relationships. The task body is then connected to the NEURA IR [24], allowing the NEURA backend to profile and map each task while carrying AMOEBA’s counter information. The resulting representation gives later compiler stages a unified view of task dependences, per-task computation, and loop-counter structure.

B. Task Classification and Profiling

Task classification and profiling identify each task’s DLP capability and estimate its latency under different tile-array sizes. This information allows orchestration to choose each task’s tile-array and the core allocation for DLP-capable tasks, thereby satisfying heterogeneous tile and core demands.

Task Classification. AMOEBA first classifies the counter domains inside each task. For every counter, the compiler identifies whether it is static-bound or symbolic-bound, following the counter semantics introduced in Sec. III-A. AMOEBA then marks a task as DLP-capable if its counter-driven computation contains at least one partitionable counter dimension. Loop-carried reduction dimensions are excluded because splitting them across replicas would require merging partial results. The compiler also records the IDs of these partitionable counters for later data partitioning.

Task Profiling. AMOEBA profiles each task under candidate tile-array allocations to estimate how efficiently the task

Algorithm 1 Throughput-Guided Task Orchestration

Input: Task graph G , task profiles P , architecture specification A .
Output: Resource allocation R , context schedule S , replica config C .

- 1: $R \leftarrow \text{InitAllocation}(G, P)$ $\triangleright$ One core and one replica per task
- 2: $S \leftarrow \text{Schedule}(G, R, A)$
- 3: $(\text{pipelineInterval}, \text{bottleneckTask}) \leftarrow \text{Analyze}(S)$
- 4: **while true do**
- 5: $(\text{bestMove}, \text{bestInterval}) \leftarrow (\emptyset, \text{pipelineInterval})$
- 6: $\text{Moves} \leftarrow \text{LargerTileArray}(\text{bottleneckTask}, P)$
- 7: $\text{Moves} \leftarrow \text{Moves} \cup \text{MoreReplicas}(\text{bottleneckTask})$
- 8: **for all** move $m \in \text{Moves}$ **do**
- 9: $R_m \leftarrow \text{ApplyMove}(R, m)$
- 10: $S_m \leftarrow \text{Schedule}(G, R_m, A)$
- 11: $(\text{interval}_m, \text{bottleneck}_m) \leftarrow \text{Analyze}(S_m)$
- 12: **if** $\text{interval}_m < \text{bestInterval}$ **then**
- 13: $(\text{bestMove}, \text{bestInterval}) \leftarrow (m, \text{interval}_m)$
- 14: **if** $\text{bestMove} = \emptyset$ **then**
- 15: **break**
- 16: $R \leftarrow \text{ApplyMove}(R, \text{bestMove})$
- 17: $S \leftarrow \text{Schedule}(G, R, A)$
- 18: $(\text{pipelineInterval}, \text{bottleneckTask}) \leftarrow \text{Analyze}(S)$
- 19: $C \leftarrow \text{GenerateRuntimeReplicaConfig}(G, R, A)$
- 20: **return** (R, S, C)

can use different numbers of tiles. For each task, the compiler maps its intra-task computation to single-array and composed-array candidates through the NEURA backend. For symbolic-bound counters, AMOEBA uses representative bound values of the task for profiling. The profiling result records each task’s latency under each candidate tile-array allocation. AMOEBA saves the profiling results and reuses them in later compilation runs to reduce compilation time.

C. Throughput-Guided Task Orchestration

Algorithm 1 shows AMOEBA’s throughput-guided orchestration algorithm, which jointly determines resource allocation and context scheduling. It starts with one core and one replica per task (line 1), schedules the task graph, and analyzes the scheduling result to obtain the task pipeline interval and the bottleneck task (lines 2-3). The pipeline interval is the output interval of the scheduled task pipeline, where a smaller interval indicates higher throughput. In each iteration, AMOEBA generates candidate resource-allocation moves for the bottleneck task (lines 6-7). *LargerTileArray* selects the next profiled tile-array allocation, while *MoreReplicas* adds one replica when

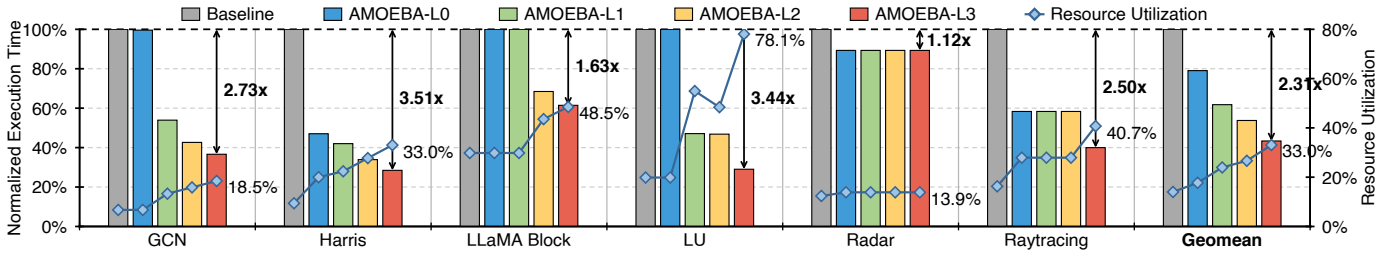


Fig. 7. Ablation Study of AMOEBA Adaptive Multi-Task Execution — Execution times are normalized to the baseline for each benchmark, and the rightmost group reports the geometric mean (geomean) across all benchmarks. AMOEBA-L0 enables context-based task-level spatio-temporal execution; L1 adds elastic tile-array composition; L2 adds static data-partitioned execution across cores; and L3 (full system) adds runtime task pipeline rebalancing with a 10-round window. The line shows resource utilization on the right axis.

TABLE II
EVALUATION BENCHMARKS.

Benchmark	Domain	Dataset	#Tasks
GCN [11]	Graph Learning	150 ENZYMES [3] graphs for inference	28
Harris [16]	Computer Vision	150 RGB image blocks	24
LLaMA Block [50]	LLM Inference	150 mixed-length requests	10
LU Decomposition [47]	Linear Algebra	150 varied-size matrices	9
Radar [38]	Signal Processing	150 FMCW radar frames	21
Raytracing [47]	Computer Graphics	150 pixel batches from a 600×400 scene	27

the bottleneck task is DLP-capable. AMOEBA speculatively applies each move and re-runs scheduling to evaluate its pipeline interval (lines 9-11). It commits the best move if the move reduces the current interval (lines 12-16). Otherwise, the search terminates (lines 14-15). After committing a move, AMOEBA reschedules the task graph and repeats the analysis on the new scheduling result (lines 17-18). Finally, AMOEBA outputs the resource allocation, context schedule, and runtime replica configurations for task-level spatio-temporal execution and runtime pipeline rebalancing (line 20).

VI. EXPERIMENTS

We present a comprehensive evaluation to demonstrate the effectiveness of AMOEBA. Our evaluation first shows that AMOEBA enables adaptive multi-task execution and improves both performance and resource utilization in Sec. VI-B. It then demonstrates that AMOEBA outperforms SOTA hierarchical SDAs in Sec. VI-C and scales across different core-array sizes in Sec. VI-D. Finally, we quantify AMOEBA’s hardware overhead in Sec. VI-E.

A. Implementation and Experimental Methodology

Implementation. We implement the AMOEBA architecture in PyMTL3 [18], [46] and generate synthesizable Verilog. We evaluate hardware overhead using Synopsys Design Compiler [42] and Cadence Innovus [4] with the TSMC 22nm ULL [51] library. The AMOEBA runtime is implemented in Python and runs on the host CPU. The AMOEBA compiler is implemented in 12K lines of C++ code on top of the MLIR [23] infrastructure. We use MLIR to define the

AMOEBA task IR, which implements our resource-aware multi-task representation. We also build the analysis and transformation passes on MLIR.

Benchmarks. We collect a diverse suite of multi-task applications covering a wide spectrum of task characteristics and resource demands, as summarized in Table II. These benchmarks include tasks with different tile-array demands, DLP-capable tasks whose latency can vary across input instances, and task graphs whose total resource demand may exceed the available cores. After being lowered to AMOEBA’s resource-aware multi-task representation, the six benchmarks contain 119 tasks in total, with each benchmark containing 9 to 28 tasks. For each benchmark, we evaluate 150 input instances sampled from benchmark-specific input sources.

Evaluated Architectures. We build a cycle-level simulator that models both inter- and intra-core execution while accounting for configuration loading and CPU-SDA communication time. AMOEBA uses a 4×4 core array for main evaluation, where each core contains a 2×2 tile array and a 32KB four-bank scratchpad memory. The **baseline** has the same fabric size and uses a conventional fixed one-core-per-task execution model with block-level time multiplexing when the task graph requires more cores than available. We also model SOTA hierarchical SDAs, including Neura [45], SARA [57], and TaskStream [5], in the simulator. We normalize their computing fabrics to the same size to compare the performance.

B. AMOEBA Enables Adaptive Multi-Task Execution

To quantify AMOEBA’s adaptive multi-task execution, we perform an ablation study with incrementally enhanced AMOEBA configurations:

- **AMOEBA-L0 (Task-Level Spatio-Temporal Execution):** This configuration enables context-based task-level spatio-temporal execution. It still uses a fixed tile array and one replica per task.
- **AMOEBA-L1 (L0 + Tile-Array Composition):** This configuration adds elastic tile-array composition with hierarchical and distributed loop control. Tasks can compose neighboring tile arrays into a larger logical tile array to satisfy higher tile demand.
- **AMOEBA-L2 (L1 + Static Data-Partitioned Execution):** This configuration supports static data-partitioned

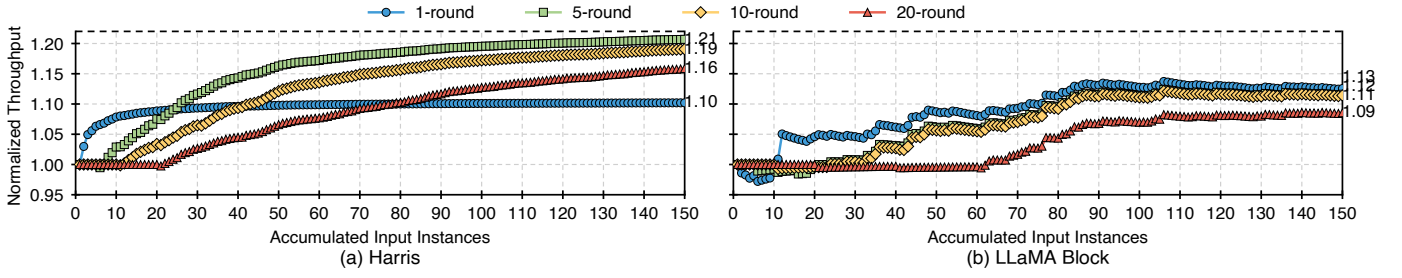


Fig. 8. Runtime Task Pipeline Rebalancing under Different Windows — Normalized accumulated throughput of Harris and LLaMA Block over 150 input instances. Throughput is normalized to AMOEBA-L2 (i.e., without runtime rebalancing). An N -round window triggers task pipeline rebalancing after every N input instances.

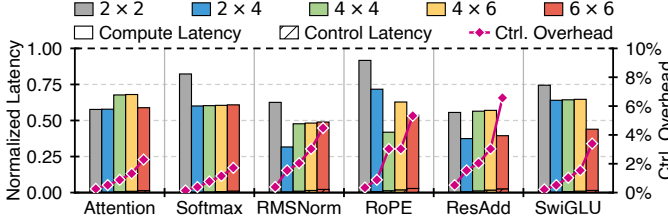


Fig. 9. Efficiency of Elastic Tile-Array Composition — Representative tasks from Fig. 2(a) are evaluated under tile-array sizes composed from neighboring 2×2 AMOEBA cores. Latency is normalized to each task’s conventional 2×2 execution latency under CPU-managed loop control. Bars breakdown normalized latency into compute and control latency, while the line shows control overhead, defined as control latency/total latency.

execution for DLP-capable tasks. The compiler assigns fixed replicas for a task, but the core allocation does not change across different inputs.

- **AMOEBA-L3 (Full System):** This configuration is the full AMOEBA system. It adds runtime support for throughput-guided task pipeline rebalancing, allowing the runtime to dynamically adjust task replicas.

Performance and Utilization. Fig. 7 shows normalized execution time and resource utilization, with all execution times normalized to the baseline. For AMOEBA-L3, the runtime uses a 10-round rebalancing window (i.e., triggering pipeline rebalancing every 10 input instances). AMOEBA-L3 achieves a geomean speedup of $2.31\times$ over the baseline. Workloads benefit differently from AMOEBA’s adaptive multi-task execution, depending on their resource demands. For example, GCN and LU benefit significantly from tile-array composition and data-partitioned execution, while Harris and Raytracing gain from both task-level spatio-temporal execution and data-partitioned execution. Radar shows limited improvement because its bottleneck is split across two balanced pipeline paths, making extra resources less effective at improving performance. AMOEBA-L3 also improves geomean resource utilization by $2.36\times$ over the baseline.

Throughput under Runtime Task Pipeline Rebalancing.

Fig. 8 shows the normalized accumulated throughput of Harris and LLaMA Block under different runtime profiling windows,

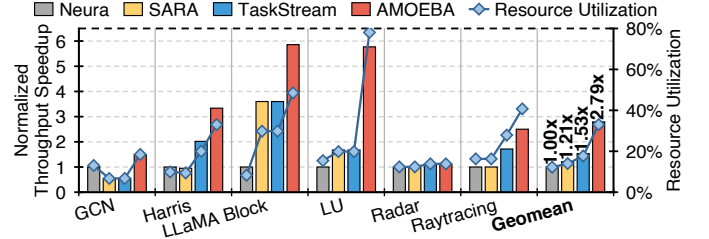


Fig. 10. Comparison with SOTA Hierarchical SDAs — Throughput speedups are normalized to Neura. AMOEBA is compared with Neura, SARA, and TaskStream under a normalized computing fabric size. The line shows resource utilization on the right axis.

with throughput normalized to AMOEBA-L2 (i.e., without runtime rebalancing). Runtime pipeline rebalancing improves throughput by up to $1.21\times$ for Harris and $1.13\times$ for LLaMA Block over AMOEBA-L2. The profiling-window size controls how often runtime rebalancing is triggered. For Harris, a 1-round window triggers rebalancing too frequently, so the overhead offsets part of the throughput gain. For LLaMA Block, faster rebalancing brings enough improvement to outweigh this overhead, making shorter windows more effective.

Efficiency of Elastic Tile-Array Composition. Fig. 9 evaluates the same representative tasks as Fig. 2(a), with latency normalized to each task’s 2×2 execution latency under the conventional CPU-managed loop control baseline described in Sec. II-C. Larger tile arrays are composed from neighboring 2×2 AMOEBA cores. AMOEBA uses counter-driven hierarchical and distributed loop control for composed-array execution. Compared with the conventional baseline, AMOEBA reduces geomean execution latency by $1.32\times$ across all evaluated tasks and tile-array sizes. It also reduces geomean control overhead by $6.60\times$. AMOEBA keeps the control overhead below 6.6% of total latency, showing that larger logical tile arrays can be composed without making loop control a major bottleneck.

C. AMOEBA Outperforms SOTA Hierarchical SDAs

Fig. 10 compares AMOEBA (i.e., AMOEBA-L3) with representative SOTA hierarchical SDAs, with all throughput

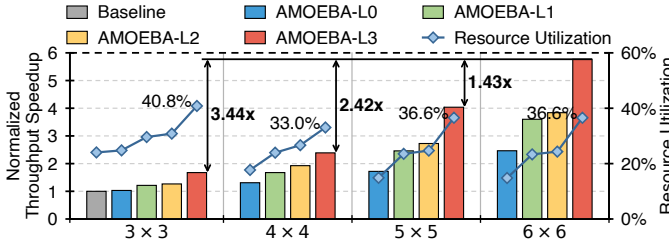


Fig. 11. Scalability across Core-Array Sizes — Core-array size scales from 3×3 to 6×6 , while each core remains a 2×2 tile array. Throughput speedups are normalized to the 3×3 baseline architecture, and the line shows AMOEBA-L3 resource utilization on the right axis.

speedups normalized to Neura. AMOEBA achieves geomean speedups of $2.79\times$ over Neura, $2.31\times$ over SARA, and $1.82\times$ over TaskStream. It also improves geomean resource utilization to 33.0%, compared with 12.2% for Neura, 14.0% for SARA, and 17.7% for TaskStream. Existing hierarchical SDAs mainly use resources at a fixed granularity, so additional tiles or cores are not always assigned to the tasks that determine the task-pipeline bottleneck. AMOEBA instead adapts tile-array size, task replicas across cores, and task-level temporal reuse according to each task’s resource demand, allowing more available resources to translate into higher performance.

D. AMOEBA is Scalable

Fig. 11 shows geomean throughput speedup and resource utilization across all evaluated benchmarks as the core array scales from 3×3 to 6×6 . Each core remains a 2×2 tile array. Throughput speedups are normalized to the 3×3 baseline architecture. AMOEBA-L3 on the 6×6 fabric improves throughput by $3.44\times$ over its 3×3 configuration, $2.42\times$ over its 4×4 configuration, and $1.43\times$ over its 5×5 configuration. This shows that AMOEBA can effectively use the additional cores exposed by a larger fabric. Meanwhile, AMOEBA-L3 maintains resource utilization between 33.0% and 40.8% across all evaluated fabric sizes, showing that AMOEBA scales performance while sustaining efficient resource use.

E. Hardware Overhead

VII. RELATED WORK

Multi-Task Execution on Hierarchical SDAs. Hierarchical SDAs provide a natural substrate for multi-task applications by organizing compute tiles into modular cores connected through an inter-core fabric. TRIPS [39] can configure the same hardware for different forms of parallelism, but its resource partitioning is organized around execution modes and threads rather than task-level resource demands. Plasticine [35] supports nested control-flow execution across compute and memory units, but it relies on hierarchical synchronization for nested control [33], [33], which can introduce pipeline bubbles. SARA [57] improves this model with compiler-managed memory consistency, using peer-to-peer synchronization to reduce this control overhead on hierarchical SDAs. Neura [45] supports multi-task execution but treats tasks as independent

workloads rather than dependent task-pipeline. TaskStream [5] targets irregular task-parallel workloads with specialized primitives for task creation, streaming, and batching. These primitives bind dependences to specific communication patterns and scheduling constraints, which limits the flexibility for task orchestration. Fifer [31] also targets irregular workloads, but it focuses on time-multiplexing predefined pipeline stages to reduce load imbalance. Octopus [8] targets orchestrated AI workflows and dynamically schedules task blocks across clusters. However, these works mainly optimize where predefined tasks execute, rather than how many hardware resources each task should use and how the hardware should be reused over time. AMOEBA addresses this limitation by adapting the hardware resources to heterogeneous task resource demands.

Adaptive Resource Allocation on SDAs. Prior work has explored adaptive resource allocation to improve SDA utilization. Some monolithic SDAs [1], [20], [27], [55], [58] partition a flat fabric or switch execution modes to support multiple tasks. Pipeline-rebalancing approaches [43], [44], [56] target streaming tasks, dynamically reallocating a monolithic tile array across tasks to relieve bottlenecks. These techniques improve utilization, but their resource choices are confined within one flat array. Neura [45] extends adaptive allocation to hierarchical SDAs by allowing a kernel to use one or multiple cores. But it focus on a single task rather than dependent task-pipeline execution, and does not address the loop-control overhead introduced by composed tile arrays. Other hierarchical SDAs support dynamic task scheduling [5], [8], [31], but they do not decide tile-array and core resources together for each task. AMOEBA fills this gap by jointly adapting tile-array composition, task replication across cores, and temporal core reuse for heterogeneous task resource demands.

VIII. CONCLUSION

REFERENCES

- [1] T. V. Aa, M. Palkovic, M. Hartmann, P. Raghavan, A. Dejonghe, and L. Van der Perre, “A multi-threaded coarse-grained array processor for wireless baseband,” in *2011 IEEE 9th Symposium on Application Specific Processors (SASP)*, 2011, pp. 102–107.
- [2] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. K. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichii, H. Suk, S. Zhang, M. Suo, P. Tillet, X. Zhao, E. Wang, K. Zhou, R. Zou, X. Wang, A. Mathews, W. Wen, G. Chanan, P. Wu, and S. Chintala, “Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 929–947. [Online]. Available: <https://doi.org/10.1145/3620665.3640366>
- [3] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. Vishwanathan, A. J. Smola, and H.-P. Kriegel, “Protein function prediction via graph kernels,” *Bioinformatics*, vol. 21, no. suppl_1, pp. i47–i56, 2005.
- [4] Cadence, “Cadence innovus,” 2026, accessed on July 4, 2026. [Online]. Available: https://www.cadence.com/en_US/home/tools/digital-design-and-signoff/soc-implementation-and-floorplanning/innovus-implementation-system.html

- [5] V. Dadu and T. Nowatzki, "Taskstream: accelerating task-parallel workloads by recovering program structure," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 1–13. [Online]. Available: <https://doi.org/10.1145/3503222.3507706>
- [6] W. J. Dally and B. P. Towles, *Principles and practices of interconnection networks*. Elsevier, 2004.
- [7] T. De Matteis, L. Gianinazzi, J. de Fine Licht, and T. Hoefler, "Streaming task graph scheduling for dataflow architectures," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing*, ser. HPDC '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 225–237. [Online]. Available: <https://doi.org/10.1145/3588195.3592999>
- [8] J. Deng, X. Tang, Z. Yue, G. Lu, Q. Yang, J. Zhang, J. Li, C. Li, S. Wei, Y. Hu *et al.*, "Efficient orchestrated ai workflows execution on scale-out spatial architecture," *IEEE Transactions on Circuits and Systems for Artificial Intelligence*, vol. 1, no. 2, pp. 229–243, 2024.
- [9] J. Deng, X. Tang, J. Zhang, Y. Li, L. Zhang, B. Han, H. He, F. Tu, L. Liu, S. Wei, Y. Hu, and S. Yin, "Towards efficient control flow handling in spatial architecture via architecting the control flow plane," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1395–1408. [Online]. Available: <https://doi.org/10.1145/3613424.3614246>
- [10] K. Feng, T. Kong, K. Koul, J. Melchert, A. Carsello, Q. Liu, G. Nyengele, M. Strange, K. Zhang, A. Nayak, J. Setter, J. Thomas, K. Sreedhar, P.-H. Chen, N. Bhagdikar, Z. A. Myers, B. D'Agostino, P. Joshi, S. Richardson, C. Torng, M. Horowitz, and P. Raina, "Amber: A 16-nm system-on-chip with a coarse-grained reconfigurable array for flexible acceleration of dense linear algebra," *IEEE Journal of Solid-State Circuits*, vol. 59, no. 3, pp. 947–959, 2024.
- [11] M. Fey and J. E. Lenssen, "Fast graph representation learning with pytorch geometric," 2019. [Online]. Available: <https://arxiv.org/abs/1903.02428>
- [12] S. Ghosh, G. Gobieski, K. Zhang, B. Lucia, N. Beckmann, and T. Nowatzki, "Nupea: Optimizing critical loads on spatial dataflow architectures via non-uniform processing-element access," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1627–1640. [Online]. Available: <https://doi.org/10.1145/3695053.3731061>
- [13] S. Ghosh, Y. Shi, B. Lucia, and N. Beckmann, "Ripple: Asynchronous programming for spatial dataflow architectures," *Proc. ACM Program. Lang.*, vol. 9, no. PLDI, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3729256>
- [14] G. Gobieski, S. Ghosh, M. Heule, T. Mowry, T. Nowatzki, N. Beckmann, and B. Lucia, "Riptide: A programmable, energy-minimal dataflow compiler and architecture," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, pp. 546–564.
- [15] M. Hamzeh, A. Shrivastava, and S. Vrudhula, "Branch-aware loop mapping on cgras," in *Proceedings of the 51st Annual Design Automation Conference*, ser. DAC '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 1–6. [Online]. Available: <https://doi.org/10.1145/2593069.2593100>
- [16] C. Harris, M. Stephens *et al.*, "A combined corner and edge detector," in *Alvey vision conference*, vol. 15, no. 50. Manchester, UK, 1988, pp. 10–5244.
- [17] M. Jiang, Y. Zhu, H. You, C. Tan, Z. Li, J. Xu, and L. Ju, "Fhe-cgra: Enable efficient acceleration of fully homomorphic encryption on cgras," in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, ser. DAC '24. New York, NY, USA: Association for Computing Machinery, 2024. [Online]. Available: <https://doi.org/10.1145/3649329.3656536>
- [18] S. Jiang, P. Pan, Y. Ou, and C. Batten, "Pymlt3: A python framework for open-source hardware modeling, generation, simulation, and verification," *IEEE Micro*, vol. 40, no. 4, p. 58–66, Jul. 2020. [Online]. Available: <https://doi.org/10.1109/MM.2020.2997638>
- [19] M. Karunaratne, A. K. Mohite, T. Mitra, and L.-S. Peh, "Hycube: A cgra with reconfigurable single-cycle multi-hop interconnect," in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2017, pp. 1–6.
- [20] T. Kong, K. Koul, P. Raina, M. Horowitz, and C. Torng, "Hardware abstractions and hardware mechanisms to support multi-task execution on coarse-grained reconfigurable arrays," 2023. [Online]. Available: <https://arxiv.org/abs/2301.00861>
- [21] K. Koul, M. Strange, J. Melchert, A. Carsello, Y. Mei, O. Hsu, T. Kong, P.-H. Chen, H. Ke, K. Zhang, Q. Liu, G. Nyengele, A. Balasingam, J. Adivarahan, R. Sharma, Z. Xie, C. Torng, J. Emer, F. Kjolstad, M. Horowitz, and P. Raina, "Onyx: A programmable accelerator for sparse tensor algebra," in *2024 IEEE Hot Chips 36 Symposium (HCS)*, 2024, pp. 1–91.
- [22] H. Kwon, A. Samajdar, and T. Krishna, "Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects," in *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 461–475. [Online]. Available: <https://doi.org/10.1145/3173162.3173176>
- [23] C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, and O. Zinenko, "Mlir: Scaling compiler infrastructure for domain specific computation," in *2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2021, pp. 2–14.
- [24] S. Li, J. Ge, D. Tao, Z. Li, J. Liang, L. Du, J. Xu, W. Zhang, and C. Tan, "Neura: A unified and retargetable compilation framework for coarse-grained reconfigurable architectures," *Proc. ACM Program. Lang.*, vol. 10, no. PLDI, Jun. 2026. [Online]. Available: <https://doi.org/10.1145/3808285>
- [25] S. Li, C. Tan, Z. Li, J. Ge, J. Liang, H. Yang, L. Du, J. Xu, and W. Zhang, "A cluster-based distributed memory architecture for cgras," in *2026 Design, Automation & Test in Europe Conference (DATE)*, 2026, pp. 1–3. [Online]. Available: <https://doi.org/10.23919/DATE69613.2026.11539358>
- [26] Z. Li, P. Dangi, C. Yin, T. K. Bandara, R. Juneja, C. Tan, Z. Bai, and T. Mitra, "Enhancing cgra efficiency through aligned compute and communication provisioning," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 410–425. [Online]. Available: <https://doi.org/10.1145/3669940.3707230>
- [27] Z. Li, D. Wijerathne, X. Chen, A. Pathania, and T. Mitra, "Chordmap: Automated mapping of streaming applications onto cgra," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 2, pp. 306–319, 2022.
- [28] L. Liu, J. Zhu, Z. Li, Y. Lu, Y. Deng, J. Han, S. Yin, and S. Wei, "A survey of coarse-grained reconfigurable architecture and design: Taxonomy, challenges, and applications," *ACM Comput. Surv.*, vol. 52, no. 6, Oct. 2019. [Online]. Available: <https://doi.org/10.1145/3357375>
- [29] B. Mei, S. Vernalde, D. Verkest, H. De Man, and R. Lauwereins, "Dresc: A retargetable compiler for coarse-grained reconfigurable architectures," in *2002 IEEE International Conference on Field-Programmable Technology, 2002.(FPT). Proceedings.* IEEE, 2002, pp. 166–173.
- [30] W. S. Moses, L. Chelini, R. Zhao, and O. Zinenko, "Polygeist: Raising c to polyhedral mlir," in *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2021, pp. 45–59.
- [31] Q. M. Nguyen and D. Sanchez, "Fifer: Practical acceleration of irregular applications on reconfigurable architectures," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 1064–1077. [Online]. Available: <https://doi.org/10.1145/3466752.3480048>
- [32] A. Podobas, K. Sano, and S. Matsuoka, "A survey on coarse-grained reconfigurable architectures from a performance perspective," *IEEE Access*, vol. 8, pp. 146719–146743, 2020.
- [33] R. Prabhakar, D. Koeplinger, K. J. Brown, H. Lee, C. De Sa, C. Kozyrakis, and K. Olukotun, "Generating configurable hardware from parallel patterns," in *Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 651–665. [Online]. Available: <https://doi.org/10.1145/2872362.2872415>
- [34] R. Prabhakar, R. Sivaramakrishnan, D. Gandhi, Y. Du, M. Wang, X. Song, K. Zhang, T. Gao, A. Wang, X. Li, Y. Sheng, J. Brot, D. Sokolov, A. Vivek, C. Leung, A. Sabnis, J. Bai, T. Zhao, M. Gottscho, D. Jackson, M. Luttrell, M. K. Shah, Z. Chen, K. Liang, S. Jain, U. Thakker, D. Huang, S. Jairath, K. J. Brown, and K. Olukotun, "Sambanova sn40l: Scaling the ai memory wall with

- dataflow and composition of experts,” in *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '24. IEEE Press, 2024, p. 1353–1366. [Online]. Available: <https://doi.org/10.1109/MICRO61859.2024.00100>
- [35] R. Prabhakar, Y. Zhang, D. Koepfinger, M. Feldman, T. Zhao, S. Hadjis, A. Pedram, C. Kozyrakis, and K. Olukotun, “Plasticine: A reconfigurable architecture for parallel patterns,” in *Proceedings of the 44th Annual International Symposium on Computer Architecture*, ser. ISCA '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 389–402. [Online]. Available: <https://doi.org/10.1145/3079856.3080256>
- [36] J. Qin, T. Xia, C. Tan, J. Zhang, and S. Q. Zhang, “Picachu: Plug-in cgra handling upcoming nonlinear operations in llms,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 845–861. [Online]. Available: <https://doi.org/10.1145/3676641.3716013>
- [37] B. R. Rau, “Iterative modulo scheduling: an algorithm for software pipelining loops,” in *Proceedings of the 27th Annual International Symposium on Microarchitecture*, ser. MICRO 27. New York, NY, USA: Association for Computing Machinery, 1994, p. 63–74. [Online]. Available: <https://doi.org/10.1145/192724.192731>
- [38] M. A. Richards *et al.*, *Fundamentals of radar signal processing*. McGraw-hill New York, 2005, vol. 1.
- [39] K. Sankaralingam, R. Nagarajan, H. Liu, C. Kim, J. Huh, N. Ranganathan, D. Burger, S. W. Keckler, R. G. McDonald, and C. R. Moore, “Trips: A polymorphous architecture for exploiting ilp, tlp, and dlp,” *ACM Trans. Archit. Code Optim.*, vol. 1, no. 1, p. 62–93, Mar. 2004. [Online]. Available: <https://doi.org/10.1145/980152.980156>
- [40] N. Serafin, S. Ghosh, H. Desai, N. Beckmann, and B. Lucia, “Pipestitch: An energy-minimal dataflow architecture with lightweight threads,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1409–1422. [Online]. Available: <https://doi.org/10.1145/3613424.3614283>
- [41] G. Sohn, G. Zhang, K. Hossfeld, J. Kim, N. Sobotka, N. Zhang, O. Hsu, and K. Olukotun, “Streaming tensor programs: A streaming abstraction for dynamic parallelism,” in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1912–1932. [Online]. Available: <https://doi.org/10.1145/3779212.3790229>
- [42] Synopses, “Synopsys design compiler,” 2026, accessed on July 4, 2026. [Online]. Available: <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/design-compiler.html>
- [43] C. Tan, N. B. Agostini, T. Geng, C. Xie, J. Li, A. Li, K. J. Barker, and A. Tumeo, “Drips: Dynamic rebalancing of pipelined streaming applications on cgras,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 304–316.
- [44] C. Tan, T. Geng, C. Xie, N. B. Agostini, J. Li, A. Li, K. Barker, and A. Tumeo, “Dynpac: Coarse-grained, dynamic, and partially reconfigurable array for streaming applications,” in *2021 IEEE 39th International Conference on Computer Design (ICCD)*, 2021, pp. 33–40.
- [45] C. Tan, M. Jiang, Y. Sun, R. Yin, Y. Ou, Q. Zhong, L. Ju, and J. Zhang, “Neura: A unified framework for hierarchical and adaptive cgras,” in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1285–1300. [Online]. Available: <https://doi.org/10.1145/3779212.3790193>
- [46] C. Tan, D. Patil, A. Tumeo, G. Weisz, S. Reinhardt, and J. Zhang, “Vecpac: A vectorizable and precision-aware cgra,” in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 1–9.
- [47] C. Tan, C. Xie, A. Li, K. J. Barker, and A. Tumeo, “Opencgra: An open-source unified framework for modeling, testing, and evaluating cgras,” in *2020 IEEE 38th International Conference on Computer Design (ICCD)*. IEEE, 2020, pp. 381–388.
- [48] Torch-MLIR, “Torch-mlir,” 2026, accessed on June 30, 2026. [Online]. Available: <https://github.com/llvm/torch-mlir>
- [49] C. Torng, P. Pan, Y. Ou, C. Tan, and C. Batten, “Ultra-elastic cgras for irregular loop specialization,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 412–425.
- [50] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, “Llama 2: Open foundation and fine-tuned chat models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [51] TSMC, “Comprehensive ultra-low power technology platform,” 2026, accessed on July 4, 2026. [Online]. Available: https://www.tsmc.com/english/dedicatedFoundry/technology/platform_IoT_tech_22ULL_12FFCplus_ULL
- [52] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [53] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, and T. Nowatzki, “Dsagen: Synthesizing programmable spatial accelerators,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 268–281.
- [54] H. Yang, L. Du, S. Li, and W. Zhang, “Esfa: An efficient scalable fft design framework on versal ai engine,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 45, no. 5, pp. 2321–2333, 2026.
- [55] Y. Yang, C. Xie, C. Guo, L. Liu, X. Peng, D. Liu, and Y. Peng, “Fexmo: Enabling fuse execution mode for multi-task cgras,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1236–1249. [Online]. Available: <https://doi.org/10.1145/3725843.3756019>
- [56] Y. Yang, C. Xie, R. Wang, L. Liu, X. Peng, and Y. Peng, “Multisky: Dynamic resource allocation framework for high-throughput cgra multitask execution,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 45, no. 3, pp. 1339–1351, 2026.
- [57] Y. Zhang, N. Zhang, T. Zhao, M. Vilim, M. Shahbaz, and K. Olukotun, “Sara: Scaling a reconfigurable dataflow accelerator,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1041–1054.
- [58] Q. Zhu, Y. Cao, Y. Qiu, X. Gao, W. Yin, and L. Wang, “A dynamic partial reconfigurable cgra framework for multi-kernel applications,” in *2023 International Conference on Field Programmable Technology (ICFPT)*, 2023, pp. 298–299.